

FlashTix API: Request Reference

Base URL: `http://localhost:8080` (no port is set in `application.properties`, so Spring Boot uses its default).

Send JSON bodies with the header `Content-Type: application/json`.

For protected endpoints, send `Authorization: Bearer <token>`, where the token is the text that `/api/v1/auth/login` returns.

Dates use ISO-8601 `LocalDateTime` format with no timezone, for example `2026-12-01T18:00:00`.

1. Authentication

POST `/api/v1/auth/register`

Creates a new user account.

Auth: Public

JSON request body:

Field	Type	Required	Rules / Notes
name	string	Yes	Must not be blank
email	string	Yes	Must not be blank and must be a valid email
password	string	Yes	Must not be blank
role	string	No	USER or ORGANIZER . Defaults to USER if left out or blank. Any other value is rejected.

Example body:

```
{
  "name": "Rishabh",
  "email": "rishabh@example.com",
  "password": "secret123",
  "role": "ORGANIZER"
}
```

Example response:

```
{
  "id": 1,
  "name": "Rishabh",
  "email": "rishabh@example.com",
  "role": "ORGANIZER"
}
```

POST `/api/v1/auth/login`

Logs in and returns a JWT as plain text, not JSON.

Auth: Public

JSON request body:

Field	Type	Required	Rules / Notes
email	string	Yes	Must not be blank and must be a valid email
password	string	Yes	Must not be blank

Example body:

```
{
  "email": "rishabh@example.com",
  "password": "secret123"
}
```

Example response:

```
eyJhbGciOiJIUzI1NiJ9... (raw JWT string)
```

GET /api/v1/me

Test endpoint that shows which user ID the token belongs to.

Auth: Bearer token (any role)

JSON request body: *none - do not send a body*

Example response:

Authenticated as user ID: 1

2. Venues

POST /api/v1/venues

Creates a venue. You need the returned id as the venueId when you create an event.

Auth: Bearer token, role **ORGANIZER**

JSON request body:

Field	Type	Required	Rules / Notes
name	string	Yes	Must not be blank
address	string	Yes	Must not be blank
city	string	Yes	Must not be blank
capacity	integer	Yes	Minimum 1

Example body:

```
{
  "name": "NSCI Dome",
  "address": "Worli",
  "city": "Mumbai",
  "capacity": 5000
}
```

Example response:

```
{
  "id": 1,
  "name": "NSCI Dome",
  "address": "Worli",
  "city": "Mumbai",
  "capacity": 5000
}
```

3. Events

POST /api/v1/events

Creates an event. The logged-in user becomes its organizer.

Auth: Bearer token, role **ORGANIZER**

JSON request body:

Field	Type	Required	Rules / Notes
name	string	Yes	Must not be blank
description	string	Yes	Must not be blank
startDate	datetime	Yes	Must be in the future, e.g. 2026-12-01T18:00:00
endDate	datetime	Yes	Must be in the future
capacity	integer	Yes	Minimum 1
category	string	Yes	Must not be blank
venueId	number	Yes	ID of an existing venue

Example body:

```
{
  "name": "Rock Night",
  "description": "Live rock concert",
  "startDate": "2026-12-01T18:00:00",
  "endDate": "2026-12-01T23:00:00",
  "capacity": 500,
  "category": "MUSIC",
  "venueId": 1
}
```

Example response:

```
{
  "id": 1,
  "name": "Rock Night",
  "description": "Live rock concert",
  "startDate": "2026-12-01T18:00:00",
  "endDate": "2026-12-01T23:00:00",
  "capacity": 500,
  "status": "...",
  "category": "MUSIC",
  "organizerName": "Rishabh",
  "venueName": "NSCI Dome"
}
```

PUT /api/v1/events/{id}

Updates an event. Send the full object: every field is required, the same as for create.

Auth: Bearer token, role **ORGANIZER**

Other input: Path variable id, the ID of the event

JSON request body:

Field	Type	Required	Rules / Notes
name	string	Yes	Must not be blank
description	string	Yes	Must not be blank
startDate	datetime	Yes	Must be in the future, e.g. 2026-12-01T18:00:00
endDate	datetime	Yes	Must be in the future
capacity	integer	Yes	Minimum 1
category	string	Yes	Must not be blank
venueId	number	Yes	ID of an existing venue

Example body:

```
{
  "name": "Rock Night",
  "description": "Live rock concert",
  "startDate": "2026-12-01T18:00:00",
  "endDate": "2026-12-01T23:00:00",
  "capacity": 500,
  "category": "MUSIC",
  "venueId": 1
}
```

Example response:

```
{
  "id": 1,
  "name": "Rock Night",
  "description": "Live rock concert",
  "startDate": "2026-12-01T18:00:00",
  "endDate": "2026-12-01T23:00:00",
  "capacity": 500,
  "status": "...",
  "category": "MUSIC",
  "organizerName": "Rishabh",
  "venueName": "NSCI Dome"
}
```

GET /api/v1/events

Lists all events.

Auth: Public

JSON request body: none - do not send a body

Example response:

```
[ { ...event object, same shape as above... } ]
```

GET /api/v1/events/{id}

Gets one event by ID.

Auth: Public

Other input: Path variable id

JSON request body: none - do not send a body

Example response:

```
{ ...event object, same shape as above... }
```

4. Seats

POST /api/v1/events/{eventId}/seats/generate

Generates seats for an event.

Auth: Bearer token, role **ORGANIZER**

Other input: Path variable eventId

JSON request body:

Field	Type	Required	Rules / Notes
numberOfSeats	integer	Yes	Minimum 1

Example body:

```
{
  "numberOfSeats": 50
}
```

Example response:

```
[
  { "id": 1, "seatNumber": "...", "status": "..."},
  ...
]
```

5. Bookings and Payment

POST /api/v1/bookings/hold/{seatId}

Holds a seat for the logged-in user. This is the first step of booking.

Auth: Bearer token (any role)

Other input: Path variable seatId. Optional header Idempotency-Key: <unique string>, which stops a retried request from creating a duplicate booking.

JSON request body: none - do not send a body

Example response:

```
{
  "id": 10,
  "seatNumber": "...",
  "status": "..."}
}
```

POST /api/v1/bookings/{bookingId}/pay

Pays for a held booking. The payment service is a mock. The response is plain text, not JSON.

Auth: Bearer token (any role)

Other input: Path variable bookingId, plus the **query parameter** amount (a decimal number, required).

Example: /api/v1/bookings/10/pay?amount=499.0

JSON request body: none - do not send a body

Example response:

Payment SUCCESS (ref: ...)

GET /api/v1/bookings/{bookingId}/slip

Gets the payment slip (receipt) for a booking.

Auth: Bearer token (any role)

Other input: Path variable bookingId

JSON request body: *none - do not send a body*

Example response:

```
{
  "receiptNumber": "...",
  "bookingId": 10,
  "seatNumber": "...",
  "eventName": "Rock Night",
  "issuedAt": "2026-09-29T12:00:00",
  "status": "..."
}
```

Typical Test Flow

1. Register an ORGANIZER, then log in and copy the token.
2. POST /api/v1/venues to create a venue and note its id.
3. POST /api/v1/events with that venueId and note the event id.
4. POST /api/v1/events/{eventId}/seats/generate and note a seat id.
5. Register a USER, then log in and use that token for the steps below.
6. POST /api/v1/bookings/hold/{seatId} and note the booking id.
7. POST /api/v1/bookings/{bookingId}/pay?amount=...
8. GET /api/v1/bookings/{bookingId}/slip

Values shown as "..." depend on server logic and are not fixed in the DTOs.